Hedge Fund Skills: Teaching an AI Agent the Playbooks of Professional Quants

A practical framework for packaging quant methods as agent skills that load on demand, compute in code, and compose into complete research workflows.

Abstract: **An AI agent becomes truly useful to a hedge fund when it knows how the firm does its work: which data to trust, which steps to follow, and which statistics decide whether a result holds. This paper shows how to give an agent that knowledge through agent skills, the open format in which a folder of instructions, scripts, and reference files teaches an agent a specific procedure and loads only when a task calls for it. We define the anatomy of a hedge fund skill, with a precise trigger, an ordered procedure, computation in code, a data contract, built in checks, and a fixed output. We then build a library of six skills drawn from the core playbooks of professional quants: factor risk decomposition, event studies, position sizing, regime detection, strategy validation, and earnings preparation, each with its method and mathematics. Finally, we show how skills compose into complete research workflows and how to test them so that their results stay correct as the library grows.**

Keywords: **Agent skills, AI agents, quantitative finance, hedge funds, factor risk, event study, position sizing, regime detection, strategy validation.**

I. Introduction

Man Group, the largest publicly listed hedge fund, has said that its quant equity unit uses an AI system able to generate, code, and backtest trading ideas [3]. The most interesting word in that description is not generate. It is backtest. Generating ideas requires a capable model, but backtesting them the way a firm expects requires the firm's procedures: which data to use, how to adjust for corporate actions, how to treat costs, and which statistics decide whether an idea survives. That knowledge lives in playbooks, and playbooks are what separate a senior quant from a talented beginner.

There are three ways to give an agent that knowledge, summarized in Table I. The first is a long system prompt that describes every method the team uses. It works for a handful of procedures, then crowds the context window and dilutes the instructions that matter for the task at hand. The second is to let the model reason each method out from its training. Its answers are often fluent and close, but a factor risk report computed one way on Monday and another way on Tuesday cannot be compared, audited, or trusted with capital.

The third way is agent skills. A skill is a folder of instructions, scripts, and resources that an agent discovers and loads on demand [1]. The format was released by Anthropic as an open standard and is now supported across many agent products [2]. Skills load progressively, as Fig. 1 shows: the agent keeps only a short name and description of every skill in view, reads a skill's full instructions when a task matches it, and opens its scripts and reference files only as needed. A library of a hundred skills therefore costs roughly ten thousand tokens to keep available, because each description is on the order of a hundred tokens, while the full procedure arrives only when it is used. That is what lets a library keep growing without slowing the agent down.

TABLE I
Three Ways to Teach an Agent Quant Methods

Approach	How it works	Result
Long system prompt	Every method described up front, always in context	Fits a few methods; context fills as the playbook grows
Reasoning from training	The model works out each method on the fly	Fluent answers that vary from run to run
Agent skills	Procedures load on demand; scripts compute the numbers	Consistent, auditable, and scales to a full library

This makes skills a natural home for the playbooks of a quant team. A procedure becomes a versioned folder, its arithmetic becomes tested code, and the agent carries the firm's methods the way an experienced analyst does: ready when needed and out of the way otherwise.

This paper makes four contributions. By the end, the reader will understand:

1) how agent skills work and why their structure fits quantitative work so well;
2) the anatomy of a hedge fund skill, and the six elements that make a skill precise, reproducible, and safe to compose;
3) a library of six skills built from core quant playbooks, each with its procedure, its mathematics, and its checks;
4) how skills compose into research workflows and how to test a skill library so that it stays correct over time.

Section II reviews the skill format. Section III defines the anatomy of a hedge fund skill, Section IV presents the skill library, and Section V shows how skills compose into workflows. Section VI gives a method for writing new skills, Section VII covers testing and governance, Section VIII discusses where skills create the most value, and Section IX concludes.

II. Agent Skills

A. The Format

A skill is a directory containing a file named SKILL.md, which opens with a short metadata header followed by

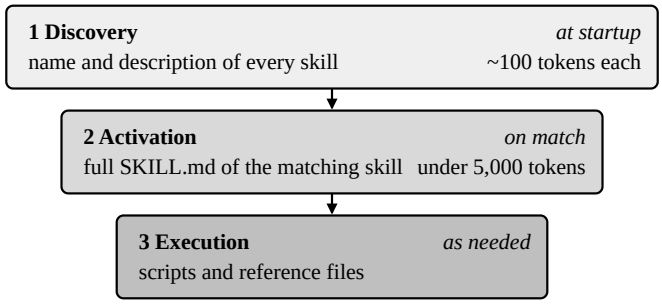


Fig. 1. Progressive disclosure of agent skills [2]. The agent keeps only short descriptions of every skill in view, reads a skill's full instructions when a task matches it, and opens its scripts and files only when a step requires them.

instructions written in plain Markdown [2]. Two header fields are required. The name identifies the skill and matches its directory, and the description states what the skill does and when to use it, in at most 1,024 characters. Optional fields record a license, environment requirements, and arbitrary metadata such as an owner and a version. Beside SKILL.md, a skill may hold a scripts directory with executable code, a references directory with detailed documentation, and an assets directory with templates and data files.

B. Progressive Disclosure

Agents load skills in three stages [2]. At discovery, only names and descriptions are loaded, roughly a hundred tokens per skill. At activation, the full SKILL.md body enters the context, and the specification recommends keeping it under five thousand tokens and five hundred lines. At execution, the agent follows the instructions, runs bundled scripts, and reads referenced files only when a step requires them. A script can run without its source code or its input data ever entering the context, which keeps large computations cheap and deterministic [1].

C. Why Quant Work Fits

Quantitative research is unusually procedural. A factor risk report, an event study, and a backtest validation each follow a sequence of steps that experienced practitioners agree on, and each produces numbers that must be computed exactly the same way every time. Skills capture both halves of that work. The instructions encode the judgment, such as which window to use and which checks to run, while the scripts encode the arithmetic, which language models should never perform from memory. The model contributes what it does best: understanding the request, choosing the right skill, and explaining the result.

III. Anatomy of a Hedge Fund Skill

A skill that will be trusted with research decisions needs more than instructions. We define six elements that every hedge fund skill should contain. Fig. 2 shows a complete example for the event study skill.

A. Trigger

The description is the only part of a skill the agent sees before deciding to use it, so it must state both the task and the situations that call for it, using the words a portfolio manager would actually use. A description such as measures how the market priced a dated event, used when asked how a stock reacted to earnings, guidance, or a rating change, is far more reliable than one that simply says event analysis.

B. Procedure

The body lists the steps in order, with the parameters a practitioner would choose: estimation windows, lookback lengths, thresholds, and the order in which inputs are resolved. Each step names the script that performs it, so the instructions read like the checklist an experienced analyst follows.

C. Computation in Code

Every number a skill reports is computed by a bundled script, never by the model. This single rule removes arithmetic errors, makes results reproducible across runs and agents, and lets the same script be tested independently of any model.

```
events/
  SKILL.md
  scripts/fit_market_model.py
  scripts/car.py
  references/methodology.md
```

```
---
name: events
description: Measures how the market priced a
  dated event such as earnings, guidance, or a
  rating change. Use when asked how a stock
  reacted to news.
metadata:
  owner: research
  version: "1.0"
---
# Event study
1. Resolve the security and the event date.
2. Fit the market model on 250 days ending
   20 days before the event:
   scripts/fit_market_model.py
3. Compute AR, CAR and t over the window:
   scripts/car.py
4. Report CAR, t, windows, and checks.
## Checks
  Windows never overlap; prices are total return.
```

Fig. 2. A complete hedge fund skill. Top: the skill folder. Bottom: its SKILL.md, with a precise trigger in the description, an ordered procedure in which each step names the script that performs it, and the checks the output must pass.

D. Data Contract

A skill states exactly what data it expects: identifiers, frequency, adjustments, and the rule that every input is point in time, meaning it was available at the moment being analyzed. Scripts reject inputs that violate the contract with clear messages, so problems surface at the start of a run rather than in the conclusions.

E. Checks

Each skill carries invariants that must hold for its output to be valid, such as risk contributions that sum to total risk or estimation windows that never overlap the event. The agent runs the checks before reporting and states clearly when one does not pass.

F. Output

Finally, a skill returns a fixed structure: the headline numbers, the parameters used, the data window, and the check results. A fixed output is what allows skills to feed one another, which Section V builds on.

IV. The Skill Library

Table II summarizes six skills drawn from the core playbooks of professional quants. Together they cover the questions a research desk asks most often: where risk comes from, how the market priced an event, how large a position should be, what kind of market is in force, whether a result is real, and what is expected from the next earnings release.

A. Factor Risk Decomposition

This skill answers where the risk of a portfolio comes from. It uses a factor model in which the covariance of returns is $\Sigma = B\,\Sigma_f B^\top + D$, with factor exposures B, factor

TABLE II
A Library of Six Hedge Fund Skills

Skill	Use when asked	Core procedure	Math	Output
Factor risk	Where does the risk in this portfolio come from?	Build the factor covariance, decompose total volatility by position and by factor	(1)	Volatility and contributions that sum to it
Event study	How did the market price this news?	Fit a market model before the event, measure abnormal returns over the window	(2), (3)	CAR with its t statistic
Position sizing	How large should this position be?	Fractional Kelly from expected return and risk, scaled to a volatility target	(4), (5)	Weight with its inputs and target
Regime detection	What kind of market are we in?	Relative features to regime probabilities, blended exposure	(6)	Probability per regime and target exposure
Strategy validation	Is this backtest result real?	Probabilistic Sharpe ratio against the best of N trials	(7), (8)	PSR and deflated Sharpe ratio
Earnings preparation	What is expected from this earnings release?	Implied move from the straddle versus past release day moves	(9)	Implied and historical moves

Every number in the output column is computed by a bundled script; the agent selects the skill, follows its procedure, and explains the result.

covariance Σ_f, and specific variances D [4]. For portfolio weights w, total volatility and the contribution of each position are

$$\sigma_p = \sqrt{w^\top \Sigma\, w}, \qquad \mathrm{RC}_i = \frac{w_i\,(\Sigma w)_i}{\sigma_p} \tag{1}$$

and the contributions sum exactly to σ_p, which the skill verifies as its main check. The same decomposition applied to factor exposures reports how much risk comes from each factor and how much is specific to individual names. Because each contribution is the weight times the marginal risk $(\Sigma w)_i/\sigma_p$, the same output tells the desk which trades would lower risk most efficiently.

B. Event Study

This skill measures how the market priced a dated event. It estimates a market model over a window that ends well before the event, so the reaction being measured never shapes the estimate of normal returns, then computes abnormal returns and their cumulative sum over the event window [5]:

$$\mathrm{AR}_t = R_t - (\hat{\alpha} + \hat{\beta} R_{m,t}), \qquad \mathrm{CAR} = \sum_{t=t_1}^{t_2} \mathrm{AR}_t \tag{2}$$

The significance of the reaction follows from the standard deviation of abnormal returns in the estimation window, $\hat{\sigma}_{\mathrm{AR}}$, and the length L of the event window:

$$t_{\mathrm{CAR}} = \frac{\mathrm{CAR}}{\hat{\sigma}_{\mathrm{AR}}\sqrt{L}} \tag{3}$$

C. Position Sizing

This skill turns an expected return and a risk estimate into a position size. Its foundation is the Kelly criterion [6], which maximizes long run growth. For an asset with expected excess return μ and variance σ^2, the growth rate of a fraction f invested and the fraction that maximizes it are

$$g(f) = f\mu - \tfrac{1}{2} f^2 \sigma^2, \qquad f^* = \frac{\mu}{\sigma^2} \tag{4}$$

Investing half of f^* delivers three quarters of the maximum growth with half the volatility, which is why professionals favor fractional Kelly when their estimates are uncertain. The skill then scales the position toward a target volatility σ^*, so that the risk it carries stays consistent as volatility rises and falls [7]:

$$w_t = \frac{\sigma^*}{\hat{\sigma}_t} \tag{5}$$

D. Regime Detection

This skill reports what kind of market is in force. Because a ratio of volatilities means the same thing for an equity index as for a currency, it relies on relative measures. It computes relative features such as the ratio of short to long window volatility, trend strength, average cross asset correlation, and drawdown, and returns a probability for each regime using only information available at time t [8]:

$$\pi_t(k) = P(S_t = k \mid \mathcal{F}_t), \qquad w_t = \sum_k \pi_t(k)\, w_k \tag{6}$$

where w_k is the target exposure for regime k, so exposure blends smoothly as the probabilities change.

E. Strategy Validation

This skill decides whether a backtest result reflects skill. Its first measure is the probabilistic Sharpe ratio [10], the probability that the true Sharpe ratio exceeds a threshold SR^* given an observed ratio $\widehat{\mathrm{SR}}$ over T observations with skewness γ_3 and kurtosis γ_4:

$$\mathrm{PSR} = \Phi\left(\frac{(\widehat{\mathrm{SR}} - \mathrm{SR}^*)\sqrt{T-1}}{\sqrt{1 - \gamma_3 \widehat{\mathrm{SR}} + \frac{\gamma_4 - 1}{4}\widehat{\mathrm{SR}}^2}} \right) \tag{7}$$

Because research tests many variations, the skill sets the threshold to the Sharpe ratio the best of N independent trials would reach by chance [9]:

$$\begin{aligned} \mathrm{SR}_0 = \sqrt{V}\,\Big(& (1-\gamma)\, Z^{-1}\!\big(1 - \tfrac{1}{N}\big) \\ & + \gamma\, Z^{-1}\!\big(1 - \tfrac{1}{Ne}\big) \Big) \end{aligned} \tag{8}$$

where V is the variance of Sharpe ratios across trials, $\gamma \approx 0.5772$ is the Euler and Mascheroni constant, and Z^{-1} is the inverse standard normal distribution. Evaluating (7) at this threshold gives the deflated Sharpe ratio. The skill therefore requires the number of trials as an input, making honest record keeping part of the procedure.

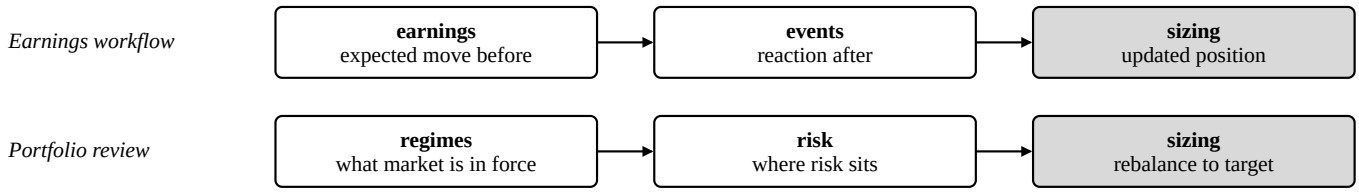


Fig. 3. Skills compose into workflows because each returns a fixed structure. The output of every step becomes the input of the next, while each number is computed by the script inside the skill that produced it.

F. Earnings Preparation

This skill prepares for a scheduled earnings release. Under a normal approximation, the price of an at the money straddle expiring after the release approximates the expected absolute move of the underlying:

$$\mathbb{E}\left[\,|S_T - S_0|\,\right] \approx C_{\mathrm{ATM}} + P_{\mathrm{ATM}} \tag{9}$$

The skill reports this implied move alongside the company's history of realized moves on past release days, and after the release it hands the event to the event study skill, so the desk sees what was expected and what actually happened.

V. Composing Skills Into Workflows

Because every skill returns a fixed structure, the output of one becomes the input of another, and complete research workflows emerge from a small library. Fig. 3 shows two of them. In the earnings workflow, the earnings preparation skill sets expectations before the release, the event study skill measures the reaction afterward, and the position sizing skill turns the updated view into a size. In the portfolio review workflow, regime detection sets the context, factor risk decomposition shows where risk sits, and position sizing rebalances toward the target. A third workflow governs new strategies: strategy validation confirms that a backtest result is more than the best of many trials, factor risk decomposition shows how the strategy would change the portfolio's exposures, and position sizing sets its starting allocation. Each workflow mirrors a process a real desk already runs, now executed the same way every time.

Algorithm 1 describes how the agent runs such a workflow. It plans from the skill descriptions alone, activates each skill when its step arrives, runs the bundled scripts, verifies the checks, and passes the structured output forward. The model reasons about the sequence and explains the results, while every number travels through tested code. Composition is also where the library compounds: each new skill adds not one capability but a new step that every existing workflow can use.

Algorithm 1: Running a skill workflow

Require: request q; skill library $\mathcal{S}$
plan the workflow for q from skill descriptions
for each step s in the plan **do**
  activate the skill for s; read its SKILL.md
  check inputs against the data contract
  run the bundled scripts on point in time data
  if any check fails **then**
    report the failed check and stop this branch
  end if
  pass the structured output to the next step
end for
return results, skill versions, and explanation

VI. Writing a Skill

The most valuable skills begin as procedures a practitioner performs by hand. Turning one into a skill follows five steps.

1) Capture the procedure. Have the person who does the task walk through it once, recording every choice: data, windows, adjustments, thresholds, and presentation.
2) Write the trigger. Draft the description from the questions people actually ask, in their own words, and include the situations in which the skill applies.
3) Move the arithmetic into scripts. Every calculation becomes a small script with a clear input and output, written so that it can run and be tested without any model. The same scripts then serve analysts directly as well.
4) Add golden cases and checks. Build an input with a known answer, and write the invariants the output must satisfy.
5) Test the trigger, then ship. Send realistic requests to the agent, confirm it selects the skill, assign an owner and a version, and add the skill to the library.

A first skill typically takes a single focused session to build, and each one after it is faster, because scripts, data contracts, and checks are shared across the library. Keeping the SKILL.md body short and moving long explanations into a references file keeps activation fast, exactly as the format recommends [2].

VII. Testing and Governance

A. Golden Cases

Each script is tested on cases with known answers, as Table III lists. Synthetic returns with a planted abnormal return must produce that return in the event study; a portfolio with a known covariance must reproduce its volatility exactly; a Kelly fraction must match the closed form in (4). These tests run whenever a script changes, like any production test suite.

B. Trigger Tests

A skill that never activates is a missing capability. A second suite sends realistic requests, phrased the way portfolio managers phrase them, and checks that the agent selects the intended skill. When it does not, the fix belongs in the description, the only text the agent reads before choosing. Each phrasing that misses becomes a permanent test case.

C. Versioning and Ownership

Skills live in version control, carry an owner and a version in their metadata, and change through review, just like the code they contain. Every report records which skill version produced it, so any number can be traced to the exact procedure behind it. Because skills can run code, a firm

TABLE III
GOLDEN CASES FOR EACH SKILL

Skill	Test with a known answer
Factor risk	Contributions sum to total volatility; a known covariance reproduces its volatility exactly
Event study	A planted abnormal return in synthetic data is recovered
Position sizing	The Kelly fraction matches the closed form in (4)
Regime detection	Probabilities sum to one and use no future data
Strategy validation	Pure noise strategies are not flagged as skill
Earnings preparation	Implied move equals the straddle price on known quotes

installs them only from trusted sources and reviews the scripts in any skill it did not write itself [1].

D. Measuring the Library

Three measures summarize the health of a skill library. Trigger accuracy is the share of realistic requests for which the agent selects the intended skill. Check pass rate is the share of runs whose invariants all hold, and a sudden change usually points to a data feed rather than a script. Reproducibility confirms that the same request on the same data, run twice, returns identical numbers. Tracking all three over time shows whether the library is improving as it grows.

VIII. DISCUSSION

Skills do not create an edge on their own. What they do is make a team's methods fast, consistent, and portable. A procedure written once by the person who knows it best becomes available to every agent and every analyst, applied the same way each time, and improved in one place. That is how a skill library becomes a form of institutional memory: the playbooks of a desk, captured in a format that both people and agents can read.

The division of labor also clarifies where people matter most. Agents handle the preparation, the computation, and the first draft of the analysis. Researchers decide which questions are worth asking, write and review the skills that answer them, and make the decisions that commit capital. The quality of a library therefore reflects the judgment of the people who write it, and the best libraries are written by the practitioners who would otherwise perform the work by hand.

Because the format is open and portable [2], a library built this way is not tied to a single model or product. As models improve, the same skills run faster and are chosen more reliably, while the procedures and the tests that define correct work stay exactly as the team wrote them.

A. Where to Start

The best first skills are the tasks a team performs most often and most consistently. Factor risk decomposition and event studies are strong starting points, because they are run constantly, their methods are well established, and their results are easy to verify against golden cases. Position sizing follows naturally, since it consumes the outputs of both. Validation and regime detection come next, and they gain value as the number of strategies under research grows. Earnings preparation is an easy early win as well, since its inputs are public option quotes and every release offers a fresh check of how closely the realized move tracked its estimate, which keeps the skill honest season after season.

B. Beyond Research

The same pattern extends across a fund. Operations teams can encode trade reconciliation and corporate action handling, compliance teams can encode restricted list and exposure limit checks, and investor relations can encode the monthly performance attribution that explains returns by factor and by strategy, while risk teams can encode stress tests against historical scenarios. Each is a procedure with clear steps, exact arithmetic, and known checks, which is precisely the shape a skill was designed to hold.

C. Principles That Scale

Libraries that grow well share a few habits. Each skill answers one question, so its description never competes with a neighbor. Each procedure leaves no open choices: windows, thresholds, and data sources are fixed in writing, not decided at run time. Each script is a pure function of its inputs, so the same data yields the same numbers. And each SKILL.md stays short, with derivations and edge cases kept in its references, read only when a step needs them.

Over time, these libraries change how a fund grows. Adding a new analyst once meant months of absorbing unwritten habits; with skills, the habits are written down, versioned, and available on the first day to every person and every agent. The firms that capture their playbooks this way will find that their best methods no longer depend on who is at the desk.

IX. CONCLUSION

Professional quants are defined less by what they know than by how consistently they apply it. Agent skills give an AI agent that consistency. A precise trigger tells it when, an ordered procedure tells it how, tested code computes every number, and checks confirm the result before anyone reads it. Six skills already cover much of the daily work of a research desk, and each new one strengthens every workflow. The model brings intelligence. The skills bring the playbook.

REFERENCES

[1] Anthropic, "Equipping agents for the real world with Agent Skills." [Online]. Available: https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[2] Agent Skills, "Specification." [Online]. Available: https://agentskills.io/specification

[3] Bloomberg News, "Man Group says agentic AI is now devising quant trading signals," Jul. 2025.

[4] R. C. Grinold and R. N. Kahn, *Active Portfolio Management*, 2nd ed. New York, NY, USA: McGraw-Hill, 2000.

[5] A. C. MacKinlay, "Event studies in economics and finance," *J. Econ. Lit.*, vol. 35, no. 1, pp. 13–39, 1997.

[6] J. L. Kelly, "A new interpretation of information rate," *Bell Syst. Tech. J.*, vol. 35, no. 4, pp. 917–926, 1956.

[7] A. Moreira and T. Muir, "Volatility-managed portfolios," *J. Finance*, vol. 72, no. 4, pp. 1611–1644, 2017.

[8] J. D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," *Econometrica*, vol. 57, no. 2, pp. 357–384, 1989.

[9] D. H. Bailey and M. López de Prado, "The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality," *J. Portfolio Manage.*, vol. 40, no. 5, pp. 94–107, 2014.

[10] D. H. Bailey and M. López de Prado, "The Sharpe ratio efficient frontier," *J. Risk*, vol. 15, no. 2, pp. 3–44, 2012.